

# shared.js

Stargazer — Supabase Bridge · Reference Guide

15 sections · 319 lines · deployed to Cloudflare Pages root

## 1 Config — credentials and admin email

Supabase project URL, anon key, and admin email constant.

```
const _SB_URL      = 'https://daqxabswnrhqpjyfdbwx.supabase.co';
const _SB_ANON     = 'eyJhbGciOi..'; // JWT anon key
const _ADMIN_EMAIL = 'ayanchakrabortyall@gmail.com';
```

## 2 State — four module-level variables

All mutable state lives here; nothing else is global.

```
let sb          = null; // Supabase client instance
let currentUser = null; // logged-in user object
let _sbSaveTimer = null; // debounce handle for progress saves
let _sbSaving    = false; // guard against concurrent upserts
```

### 3 Supabase init — auto-executing async IIFE

Creates client, checks session, fires PKCE fallback at 1200 ms, registers onAuthStateChange.

```
(async function initShared() {
  sb = supabase.createClient(_SB_URL, _SB_ANON);
  window.sb = sb;

  const { data: { session } } = await sb.auth.getSession();
  if (session?.user) {
    currentUser = window.currentUser = session.user;
    await sbLoadProgress();
  }
  renderAuthUI(); loadComments();

  setTimeout(async () => { /* PKCE/mobile fallback */ }, 1200);

  sb.auth.onAuthStateChange(async (event, session) => {
    currentUser = window.currentUser = session?.user || null;
    renderAuthUI();
    if (event === 'SIGNED_IN') await sbLoadProgress();
    loadComments();
  });
})();
```

### 4 renderAuthUI() — toggles login button, user chip, comment form

Reads 5 DOM elements by ID. Calls window.\_afterRenderAuthUI(user) as extension hook.

```
function renderAuthUI() {
  const lb = document.getElementById('loginBtn');
  const ui = document.getElementById('userInfo');
  const ue = document.getElementById('userEmailDisplay');
  const fw = document.getElementById('commentFormWrap');
  const lm = document.getElementById('commentLoginMsg');
  if (!lb) return;
  if (currentUser) {
    lb.style.display = 'none';
    ue.textContent = currentUser.user_metadata?.full_name || currentUser.email;
    if (fw) fw.style.display = 'block';
  } else {
    lb.style.display = 'flex';
    if (fw) fw.style.display = 'none';
  }
  if (typeof window._afterRenderAuthUI === 'function')
    window._afterRenderAuthUI(currentUser);
}
```

## 5 doSignOut() — signs out, clears token, reloads page

Exposed as window.doSignOut so HTML onclick attributes can call it directly.

```
window.doSignOut = async function() {
  try { if (sb) await sb.auth.signOut(); } catch(e) {}
  try { localStorage.removeItem('sb-daqxabswnrhqpjyfdbwx-auth-token'); } catch(e) {}
  currentUser = window.currentUser = null;
  renderAuthUI();
  window.location.reload();
};
```

## 6 sbSaveProgress() — debounced upsert (800 ms)

Calls window.\_getProgressData() set by each page. Guards against concurrent saves with \_sbSaving flag.

```
async function sbSaveProgress() {
  if (!currentUser || !sb) return;
  if (typeof window._getProgressData !== 'function') return;
  clearTimeout(_sbSaveTimer);
  _sbSaveTimer = setTimeout(async () => {
    if (_sbSaving) return;
    _sbSaving = true;
    try {
      const data = window._getProgressData();
      await sb.from('progress').upsert(
        [{ user_id: currentUser.id, page: window._pageKey, data }],
        { onConflict: 'user_id,page' }
      );
    } finally { _sbSaving = false; }
  }, 800);
}
```

## 7 sbLoadProgress() — fetches and applies saved progress

Calls `window._applyProgressData(data)` set by each page. Fires `window._afterProgressLoad()` on success.

```
async function sbLoadProgress() {
  if (!currentUser || !sb) return;
  if (typeof window._applyProgressData !== 'function') return;
  const { data: rows } = await sb
    .from('progress').select('data')
    .eq('user_id', currentUser.id)
    .eq('page', window._pageKey).single();
  if (!rows?.data) return;
  window._applyProgressData(rows.data);
  if (typeof window._afterProgressLoad === 'function')
    window._afterProgressLoad();
}

window.sbSaveProgress = sbSaveProgress;
window.sbLoadProgress = sbLoadProgress;
window._syncProgressUp = sbSaveProgress; // backward-compatible alias
```

## 8 BLOCKED\_WORDS and SPAM\_PATTERNS — moderation lists

29 blocked words (EN + HI) and 4 regex patterns checked before every comment insert.

```
const BLOCKED_WORDS = [
  'spam', 'fuck', 'shit', 'cunt', 'bitch', 'asshole', 'bastard',
  'motherfucker', 'scam', 'buy now', 'free money', 'viagra',
  'madarchod', 'bhosdike', 'lodu', 'harami', 'chutiya', 'gaandu',
  // ... 29 total
];

const SPAM_PATTERNS = [
  /https?:\/\/\//i, // any URL
  /\b\w+\.com\b/i, // domain names
  /(.)\1{5,}/, // character repetition (aaaaaa)
  /[!?]{4,}/ // excessive punctuation
];
```

## 9 isAbusive(text) — checks text against both lists

Lowercases and strips punctuation before word-by-word comparison.

```
function isAbusive(text) {
  const lower = text.toLowerCase().replace(/^[a-z0-9\s]/g, ' ');
  for (const w of lower.split(/\s+/)) {
    if (BLOCKED_WORDS.some(b => w === b || w.includes(b))) return true;
  }
  for (const p of SPAM_PATTERNS) { if (p.test(text)) return true; }
  return false;
}
```

## 10 isRateLimited(pageKey) — max 3 comments per 5 minutes

Timestamps stored in localStorage per page key. No server round-trip needed.

```
function isRateLimited(pk) {
  const now = Date.now();
  const key = 'sg_rate_' + pk;
  const s = JSON.parse(localStorage.getItem(key) || '[]');
  const r = s.filter(t => now - t < 5 * 60 * 1000);
  if (r.length >= 3) return true;
  r.push(now);
  localStorage.setItem(key, JSON.stringify(r));
  return false;
}
```

## 11 showCommentError(msg) — inline error below comment form

Creates #commentError div lazily; auto-clears after 4 seconds.

```
function showCommentError(msg) {
  let el = document.getElementById('commentError');
  if (!el) {
    el = document.createElement('div');
    el.id = 'commentError';
    el.style.cssText = 'color:#c0504a;font-size:0.8rem;margin-top:0.4rem;';
    document.getElementById('commentFormWrap')?.appendChild(el);
  }
  el.textContent = msg;
  setTimeout(() => { if (el) el.textContent = ''; }, 4000);
}
```

## 12 loadComments() — fetches and renders comment list

Reads window.\_pageKey for page filter. Renders delete button only for owner or admin.

```
async function loadComments() {
  const page = window._pageKey || 'unknown';
  const { data } = await sb
    .from('comments')
    .select('id,user_id,user_name,text,created_at')
    .eq('page', page)
    .order('created_at', { ascending: false })
    .limit(100);
  // builds DOM nodes for each comment:
  // .comment-item > .comment-meta (name + date [+ delete btn]) + .comment-text
  const isAdmin = currentUser?.email === _ADMIN_EMAIL;
  data.forEach(c => {
    // delete btn shown only if currentUser.id === c.user_id || isAdmin
  });
}
```

## 13 deleteComment(id) — deletes by id, reloads list

Guarded by currentUser check client-side; RLS policy enforces server-side.

```
async function deleteComment(id) {
  if (!sb || !currentUser) return;
  const { error } = await sb.from('comments').delete().eq('id', id);
  if (error) { console.warn('Delete failed:', error); return; }
  await loadComments();
}
```

## 14 submitComment(pageKey) — validates and inserts

6 validation gates in order: auth check, empty, min length, max length, abuse, rate limit.

```
async function submitComment(pageKey) {
  if (!currentUser) { window.location.href = 'login.html'; return; }
  const text = document.getElementById('commentText')?.value.trim();
  if (!text) return;
  if (text.length < 3) { showCommentError('Too short.');
```

`return; }`

```
  if (text.length > 500) { showCommentError('Too long.');
```

`return; }`

```
  if (isAbusive(text)) { showCommentError('Keep it respectful.');
```

`return; }`

```
  if (isRateLimited(pageKey)) {
    showCommentError('Too many comments. Wait a few minutes.');
```

`return; }`

```
  }
  await sb.from('comments').insert([
    { user_id: currentUser.id, user_name: name, page: pageKey, text
  }]);
  await loadComments();
}
```

## 15 Comment textarea character counter — passive listener

Attached directly (no DOMContentLoaded wrapper) because shared.js loads last in body.

```
document.getElementById('commentText')
  ?.addEventListener('input', function() {
    const cc = document.getElementById('commentCharCount');
    if (cc) cc.textContent = this.value.length + ' / 500';
  });
```